DevSync

رواد مصر الرقمية

GRADUATION PROJECT

System Analysis & Design.

DevSync: Secure AI Project Collaborator

Flutter graduation project | Implementation: 16 Oct - 30 Nov 2026

TEAM OF 5

Donia Alhosin
Bishay Talaat
Yousef Mohy
Emad Hany
Saga Maher

REPORT DURATION

16 Oct 2026 - 30 Nov 2026

Six 1-week sprints + 4-day final window

ABOUT THIS DOCUMENT

System Analysis & Design

This report describes how DevSync is analysed, modelled and deployed: requirements and architecture, data model, data flow and behaviour, UI/UX, deployment, and the supporting API, testing and deployment plans. Every statement carries a status label that reflects the Flutter repository today.

Document	System Analysis & Design
Course deliverable	Part 4
Team	Donia Alhosin, Bishay Talaat, Yousef Mohy, Emad Hany, Saga Maher
Platform	Android (Free and Pro flavors, Planned)
Window	16 October 2026 – 30 November 2026
Firebase project	`devsync-36ef3`
Repository	`github.com/devDoniaAlhosin/devsync-open-source-collaboration-app`
Basis	Project Planning & Management report; Requirements Gathering report; interactive prototype (Team tab and handoff section); `app_user.dart` and `firestore.rules` for implemented data

Where to find it

Part	What it covers
Part 1	Problem statement, objectives, scope, actors, use cases, functional and non-functional requirements, software architecture (Figures 1–3)
Part 2	Database design and data modelling: ERD, logical schema, physical Firestore schema, feed contract (Figure 4)
Part 3	Data flow and system behaviour: DFDs, sequence, activity, state and class diagrams (Figures 5–14)
Part 4	UI/UX design and prototyping: screen inventory, guidelines, shared-screen handoffs
Part 5	System deployment and integration: technology stack, deployment and component diagrams (Figures 15–16)
Part 6	Additional deliverables: API documentation, testing and validation, deployment strategy
Appendices	Team and backup chain; Flutter routes today

Status labels

Label	Meaning
Implemented	In the Flutter repo today
Planned	Must-have, not built yet
If-time	Only after the 12 Nov 2026 core freeze

> Planned schema fields in this report are working names and are not yet merged into the README. Authentication, the `users/{uid}` document, owner-only Firestore rules, the design system and the stub home shell are implemented. Cloud Functions, AI matching, FCM, Free/Pro flavors, GitHub REST and request workflows are not built yet.

PART 1

Problem statement, use cases, requirements, and architecture

1.1 Problem statement

Developers who want to contribute to open source struggle to find projects that fit their **skills and real GitHub activity**. Maintainers struggle to find **relevant contributors**. Existing tools (GitHub Explore, topics, good-first-issue lists, Up For Grabs, CodeTriage) filter by **label or topic**, not by the person. They rarely give a **scored explanation** or an **in-app collaboration request with alerts**.

DevSync is a Flutter Android app that:

- Builds a developer profile from **public GitHub activity** and declared **skills**
- Scores user-to-project fit with an LLM called only from a **Cloud Function**
- Shows a **score and a one-line reason**
- Lets users **like or skip** so prompts can be measured and improved
- Supports **collaboration requests** with accept/decline and **FCM** alerts

Out of scope for 30 Nov 2026: iOS release, store publishing, in-app payments, hosting project source inside the app (the app links to GitHub), in-app chat as a committed feature.

1.2 Objectives

Objective	Description
Working matching app	Android app with AI-driven developer-to-project matching
Secure authentication	Google and email sign-in via Firebase Authentication
Collaboration	Requests with FCM alerts for new requests and status changes
Modern UX	DevSync design system, key-screen animations
Release hygiene	Obfuscated Free and Pro builds, Android App Bundle, install link
Team learning	Each member builds UI, logic, Firestore, tests, and docs for their features
Documentation	Technical docs, README, clean Git history, security-reviewed app

1.3 Scope

Must-have (protected through core freeze 12 Nov 2026): sign-in; profile + GitHub data + skills; projects (seed + add/edit); AI match feed with score and reason; like/skip; collaboration requests; FCM; core animations; Free/Pro flavors; obfuscated builds and install link; documentation.

If-time (only after 12 Nov; feature freeze 26 Nov): task board (Donia), task alerts (Yousef), prompt-quality dashboard (Emad), extra animations and a sub-2-minute walkthrough (Bishay), regression and Pro extras (Saga).

1.4 Actors

Actor	Type	Role
Contributor	Primary	Completes profile, views matches, likes/skips, sends requests, tracks status
Maintainer	Primary	Lists/edits own projects, receives/accepts/declines requests

Actor	Type	Role
DevSync Flutter app	System	Client UI, Auth/Firestore/FCM SDKs, GitHub fetch
Firebase Authentication	External	Google, email/password, email link; session
Cloud Firestore	External	Users, projects, matches cache, feedback, requests, tasks
Cloud Functions	External	LLM matching; FCM triggers; secrets
FCM	External	Push on new request, accept/decline, project updates
GitHub API	External	Public repos, languages, contributions only
Gemini Flash-Lite	External	Fit score + short reason; model name kept in config

Supervisor, the five-person team, and pilot users are stakeholders, not runtime actors.

1.5 Use-case diagram

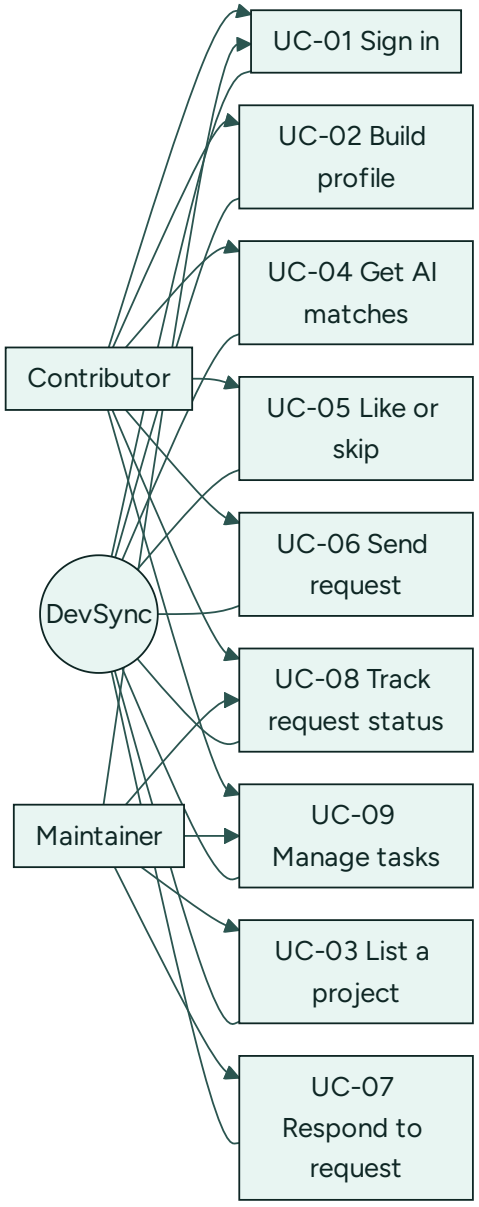


Figure 1. Use-case diagram. UC-09 is if-time.

Use-case descriptions

ID	Name	Actor	Status	Summary
UC-01	Sign in	Contributor, Maintainer	Implemented (email, Google, GitHub OAuth, email link). Reset/OTP still demo.	Create or resume a session. Persist until sign-out.
UC-02	Build profile	Contributor	Planned	Add bio and skills; fetch and cache public GitHub data.
UC-03	List a project	Maintainer	Planned	Create or edit title, description, GitHub link, needed skills. Edit own projects only.
UC-04	Get AI-ranked matches	Contributor	Planned	Call Cloud Function; show ranked feed with score and reason; skills fallback if the model fails.
UC-05	Like or skip	Contributor	Planned	Store one feedback document per user and project.
UC-06	Send request	Contributor	Planned	One request per project plus a short message.
UC-07	Respond to request	Maintainer	Planned	Accept or decline. Notify the contributor.
UC-08	Track request status	Both	Planned	Both sides see pending / accepted / declined.
UC-09	Manage tasks	Both	If-time	Shared board for an accepted pair only.

1.6 Functional requirements

ID	Area	Owner	Status	Requirement
FR-01	Auth	Donia	Implemented	Google sign-in via Firebase Auth.
FR-02	Auth	Donia	Implemented (register, email, email link). Reset is demo.	Register, email/password, password reset.
FR-03	Auth	Donia	Implemented	Session persists; sign-out.
FR-04	Auth	Donia	Planned	Delete account and profile data.
FR-05	Profile	Saga	Planned	Public GitHub: languages, repositories, contributions.
FR-06	Profile	Saga	Planned	Add, edit, remove skills and a short bio.
FR-07	Profile	Saga	Planned	Cache GitHub data; refresh in batches.
FR-08	Projects	Bishay	Planned	Store title, description, GitHub link, needed skills.
FR-09	Projects	Bishay	Planned	Maintainer edits only own projects.
FR-10	Projects	Bishay	Planned	Seed projects so the feed works on day one.
FR-11	Matching	Emad	Planned	Structured profile + versioned prompt via Cloud Function.
FR-12	Matching	Emad	Planned	Fit score and short reason.
FR-13	Matching	Emad	Planned	Rank by score; cache per user.
FR-14	Matching	Emad	Planned	Skills-based fallback if the model fails or times out.
FR-15	Feedback	Yousef	Planned	Like/skip; one feedback document per user and project.
FR-16	Matching	Emad	Planned	Versioned prompts and an evaluation set (must-have).
FR-17	Matching	Emad	If-time	Prompt-quality dashboard.
FR-18	Requests	Yousef	Planned	One request per project plus a short message.

ID	Area	Owner	Status	Requirement
FR-19	Requests	Yousef	Planned	Accept or decline.
FR-20	Requests	Yousef	Planned	Status visible to both sides.
FR-21	FCM	Yousef	Planned	FCM on new request, status change, project updates.
FR-22	FCM	Yousef	Planned	Ask notification permission at the moment of use; app works if denied.
FR-23	UX	Bishay	Planned	Implicit and TweenAnimationBuilder transitions on sign-in, feed, requests, profile.
FR-24	UX	Donia	Implemented	Follow the DevSync design system.
FR-25	Release	Yousef	Planned	Free/Pro flavors: own app ID, icon, Firebase config.
FR-26	Release	Saga	Planned	Obfuscated release and `.aab`.
FR-27	Release	Saga	Planned	Signed install link and a short install guide.
FR-28	Tasks	Donia	If-time	Two collaborators create, assign, and move tasks.
FR-29	Tasks	Yousef	If-time	FCM on task assign and status.
FR-30	Tasks	Donia	If-time	Board restricted to the accepted pair via rules.

1.7 Non-functional requirements

ID	Category	Owner	Requirement
NFR-01	Performance	Emad	Match results in 5 s or less for 95% of requests
NFR-02	Performance	Yousef	Cold start 3 s or less on test devices
NFR-03	Performance	Yousef	FCM delivered in 10 s or less for 95% of tests
NFR-04	Reliability	Saga	99% or more crash-free sessions (Crashlytics)
NFR-05	Reliability	Donia	Backend 99.5% during testing and pilot
NFR-06	Reliability	Emad / Saga	Cached data and an offline message; model fallback
NFR-07	Security	Donia	Users read/write only their own data and their requests
NFR-08	Security	Donia	No API keys in the repo; LLM key is a Cloud Function secret
NFR-09	Security	Saga	HTTPS only; obfuscated release; symbol files stored safely
NFR-10	Security	Donia / Yousef	Runtime permissions only when needed, with an explanation
NFR-11	Security	Saga	100% of the security checklist before release
NFR-12	Privacy	Saga / Donia	Only public GitHub data and declared skills; account deletion
NFR-13	UX	Donia	Touch targets 48 dp or more; contrast 4.5:1 WCAG AA
NFR-14	UX	Bishay	Sign-in to matches in under 2 minutes (If-time walkthrough)
NFR-15	UX	Each owner	Plain-language errors and a retry path
NFR-16	Maintainability	Yousef / Emad	Reviewed PRs; 70% or more coverage of matching logic; CI blocks failing tests
NFR-17	Maintainability	Each / Saga	Docs and README notes in the same PR
NFR-18	Compatibility	Saga	Two real Android devices and one emulator
NFR-19	Scalability	Donia	Pilot of 20 or more users within quota, with budget alerts

1.8 Software architecture

Style: layered Flutter client + Firebase BaaS. Not microservices. Not MVC as a server pattern. The client is screens → feature services → Firebase / HTTP SDKs. Server logic that must hide secrets (LLM key, FCM fan-out) lives in Cloud Functions.

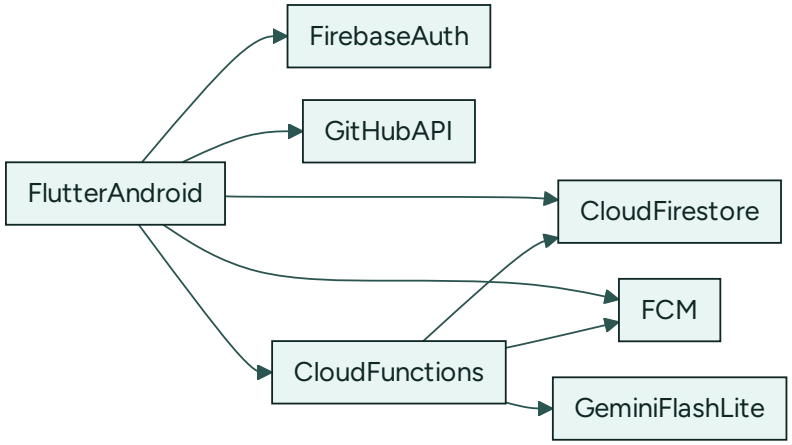


Figure 2. Target architecture. Client today talks to Auth and Firestore only.

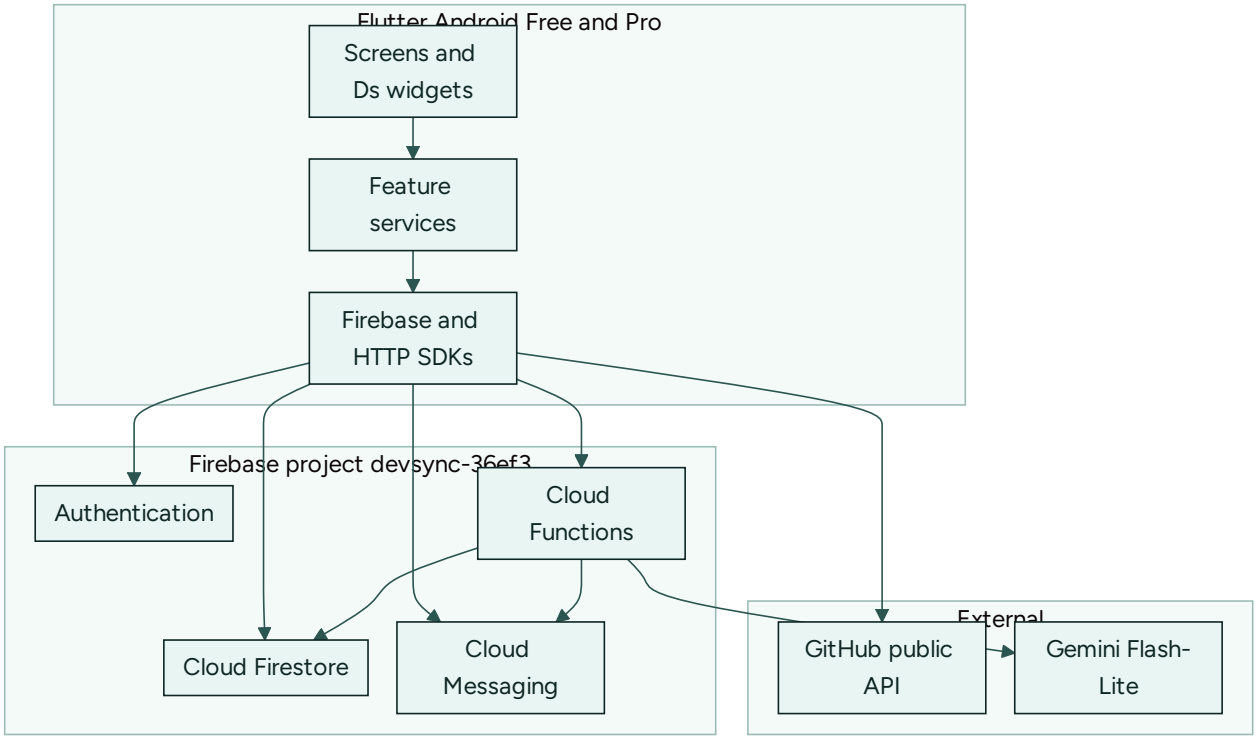


Figure 3. Layered client and BaaS. Flavors, Functions, FCM, and GitHub REST are Planned.

Implemented now: `AuthService`, `UserRepository`, `AuthGate`, design-system widgets, named routes, stub home shell. Feature folders `matching/`, `profile/`, `projects/`, `requests/`, `tasks/` are empty placeholders.

PART 2

Database design and data modeling

2.1 Entity-relationship diagram

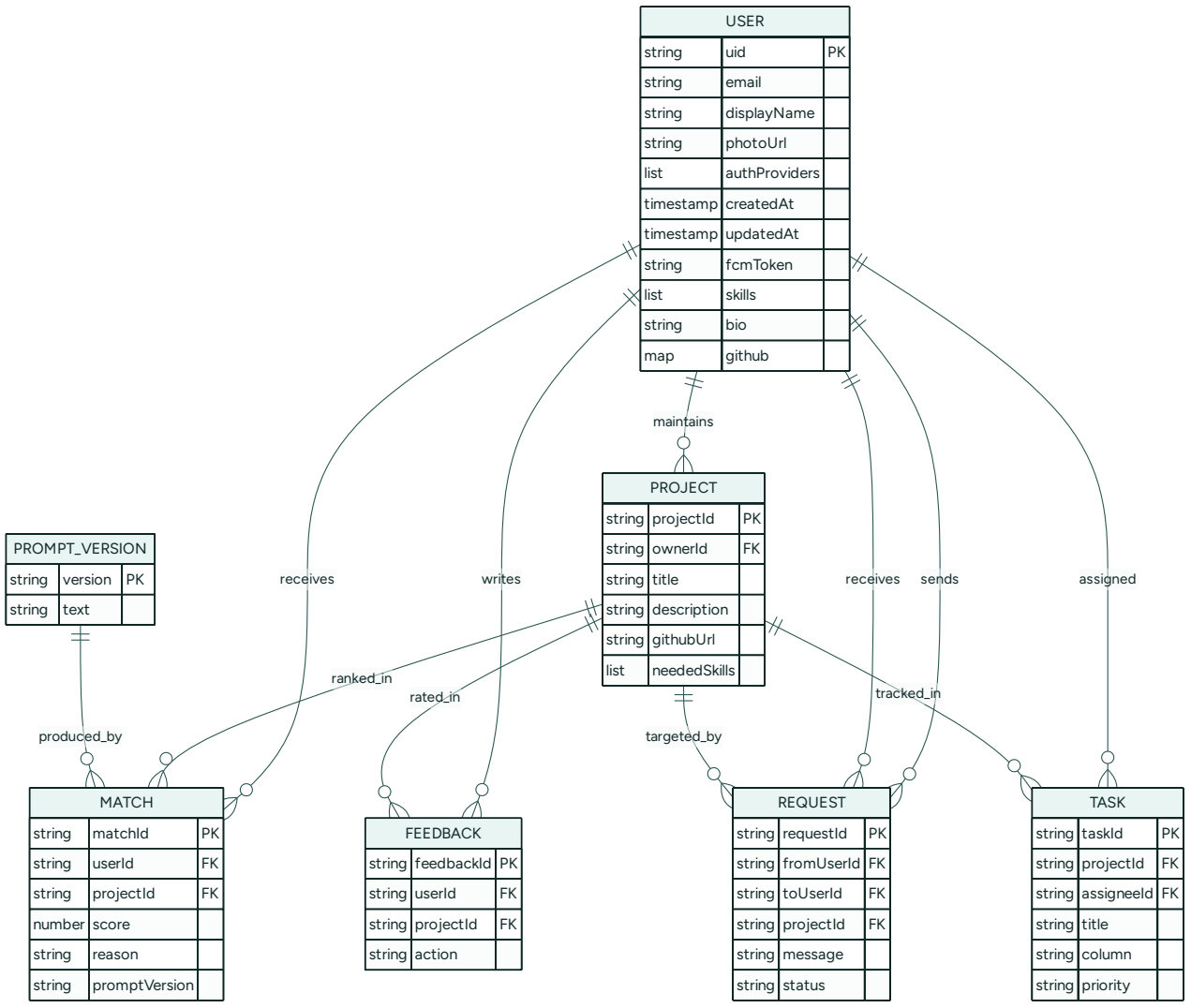


Figure 4. Conceptual ERD. `USER` identity fields are **Implemented**. `fcmToken`, `skills`, `bio`, `github`, and all other entities are **Planned**. `TASK` is **If-time**.

2.2 Logical schema

Normalize as 3NF-equivalent documents: no repeating groups of other entities inside a user; relationships use IDs.

Entity	Primary key	Foreign keys	Notes
User	`uid`	—	One document per Firebase Auth user
Project	`projectId`	`ownerId → User`	Maintainer owns the document
Match	`matchId` or `users/{uid}/matches/{projectId}`	`userId`, `projectId`	Cache; not a source of truth for projects
Feedback	one doc per `(userId, projectId)`	`userId`, `projectId`	`action` is `like` or `skip`

Entity	Primary key	Foreign keys	Notes
Request	one doc per `(fromUserId, projectId)`	`fromUserId`, `toUserId`, `projectId`	`status` is `pending`, `accepted`, or `declined`
Task	`taskId`	`projectId`, `assigneeId`	If-time; only after an accepted request
PromptVersion	`version`	—	Used by matching and evaluation

Normalization: skills stay as a string array on User and Project (they are values, not entities with their own life cycle). GitHub stats stay as a nested map on User, refreshed as a unit. Do not embed full project documents inside matches; store IDs plus the frozen feed fields.

2.3 Physical Firestore schema

Implemented — `users/{uid}` from `lib/features/auth/models/app_user.dart`:

Field	Type	Notes
`uid`	string	Same as document ID
`email`	string or null	
`displayName`	string or null	
`photoUrl`	string or null	
`authProviders`	array of string	Provider IDs such as `password`, `google.com`, `github.com`
`createdAt`	timestamp	Server timestamp on create
`updatedAt`	timestamp	Written on every upsert

Implemented rules (`firestore.rules`): authenticated user may read/write only `users/{userId}` when `request.auth.uid == userId`. No other collections are allowed yet.

Planned additions on `users/{uid}`**:** `fcmToken`, `skills` (string array), `bio`, `github` (public stats map). Exact nested GitHub keys are **not frozen**; the map is described only as public languages, repository count, and contributions.

Planned collections

Collection	Purpose	Known / frozen fields
`projects/{projectId}`	Seed and user projects	Working names: title, description, GitHub link, needed skills, owner id
`matches` (per user cache)	Ranked feed	Frozen feed item: `projectId`, `name`, `score`, `reason`, `skills[]`. Prompt version and timestamp are planned, not frozen
`feedback`	Like/skip	One document per user and project; action `like` or `skip`
`requests`	Collaboration	One per user–project; short message (prototype max 280); status `pending` / `accepted` / `declined`
`tasks`	If-time board	Columns To do / Doing / Done; assign; priority Low/Medium/High

Feed contract (frozen 23 Oct 2026 for W2 handoff):

```
{
  "projectId": "string",
  "name": "string",
  "score": 0,
  "reason": "string",
  "skills": ["string"]
}
```

The feed contract has no other required fields.

PART 3

Data flow and system behavior

3.1 Context DFD (Level 0)

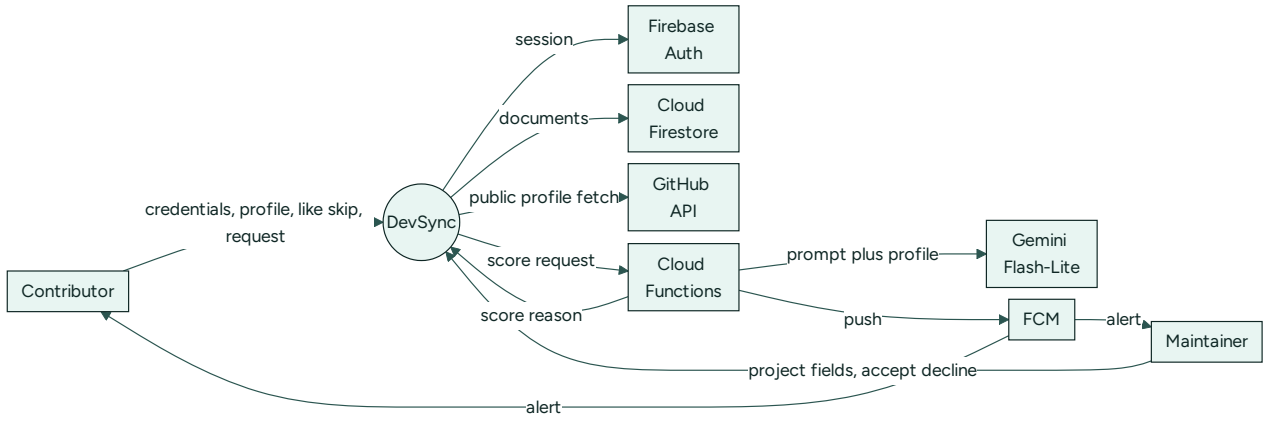


Figure 5. Context-level DFD.

3.2 Level 1 DFD

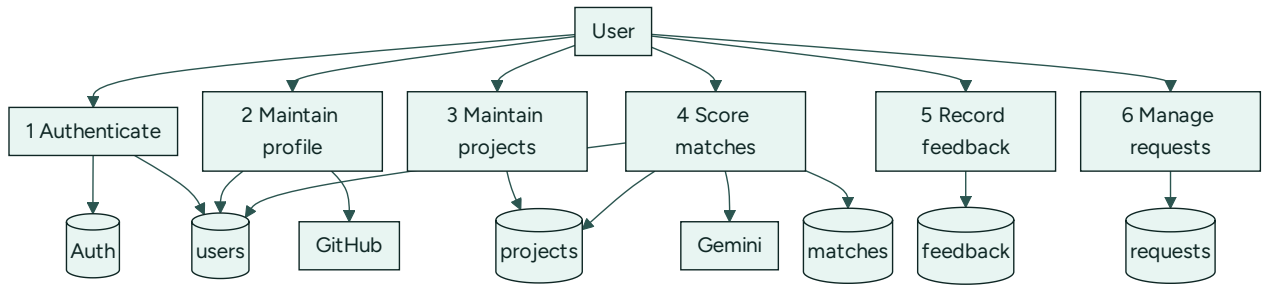


Figure 6. Level 1 DFD for auth, profile, projects, match, feedback, and requests.

3.3 Sequence — sign-in and user document

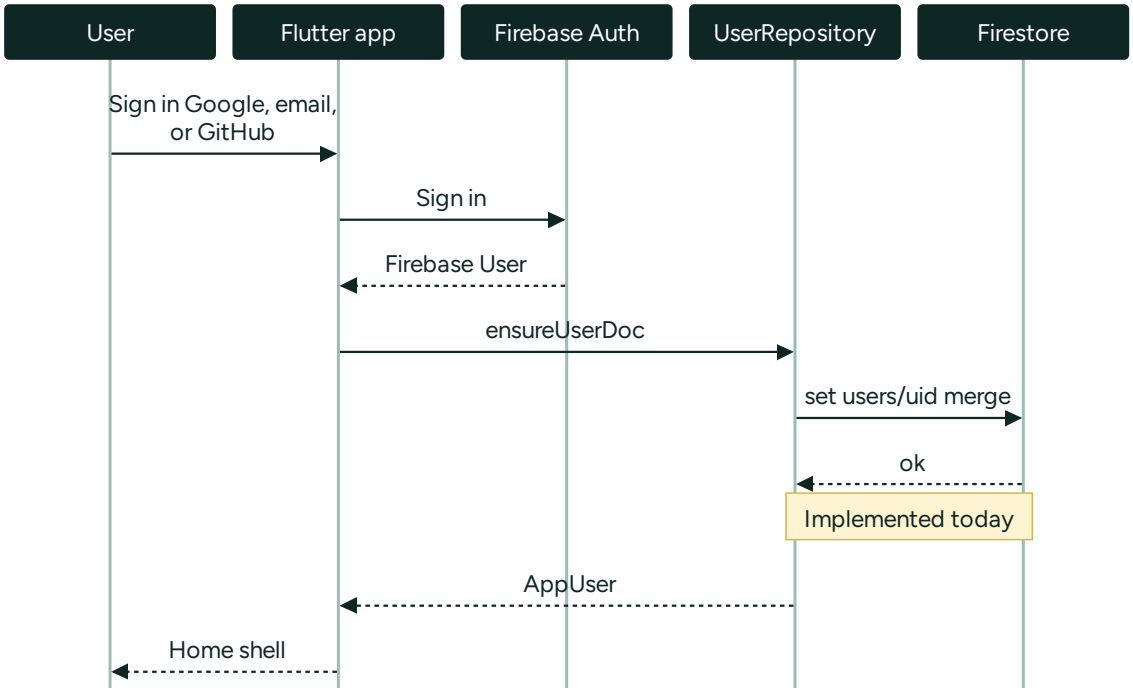


Figure 7. Sign-in sequence. Writing `fcmToken` on first sign-in is **Planned**.

3.4 Sequence — match call

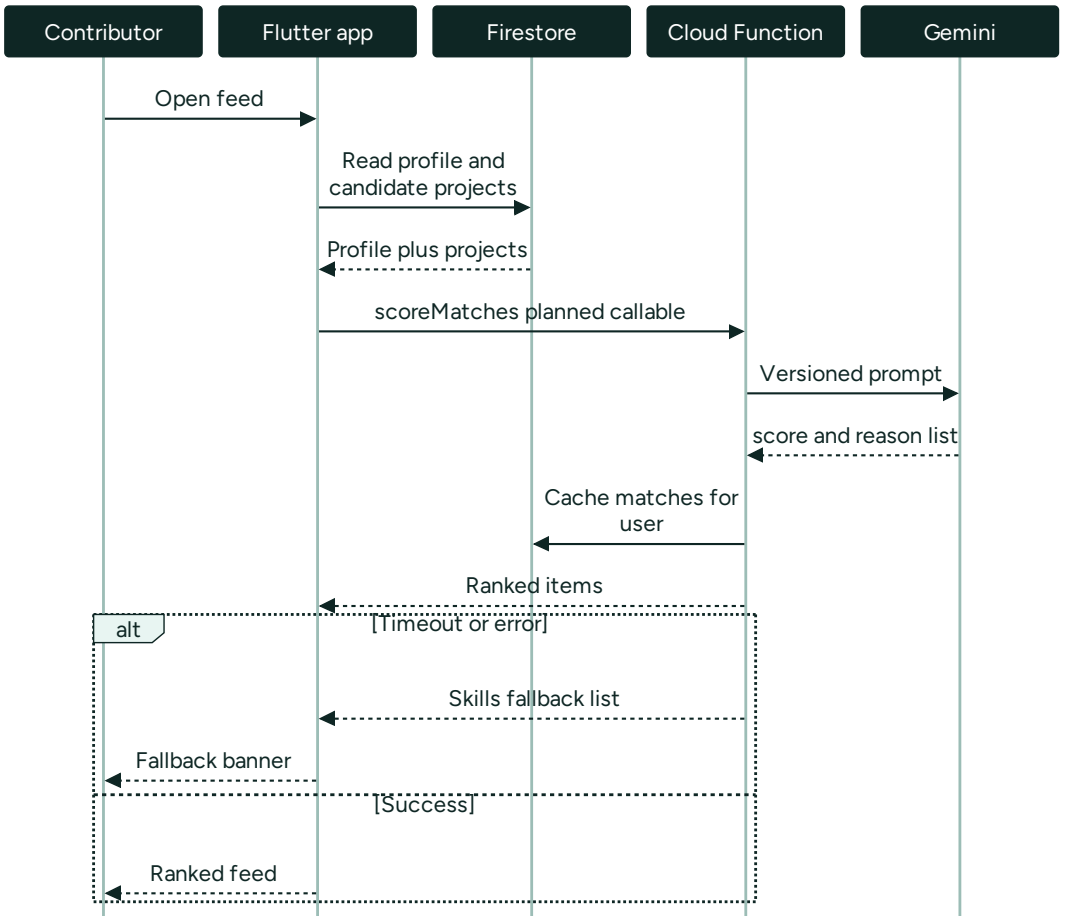


Figure 8. Matching sequence. Entire path is Planned.

3.5 Sequence — send and accept request plus FCM

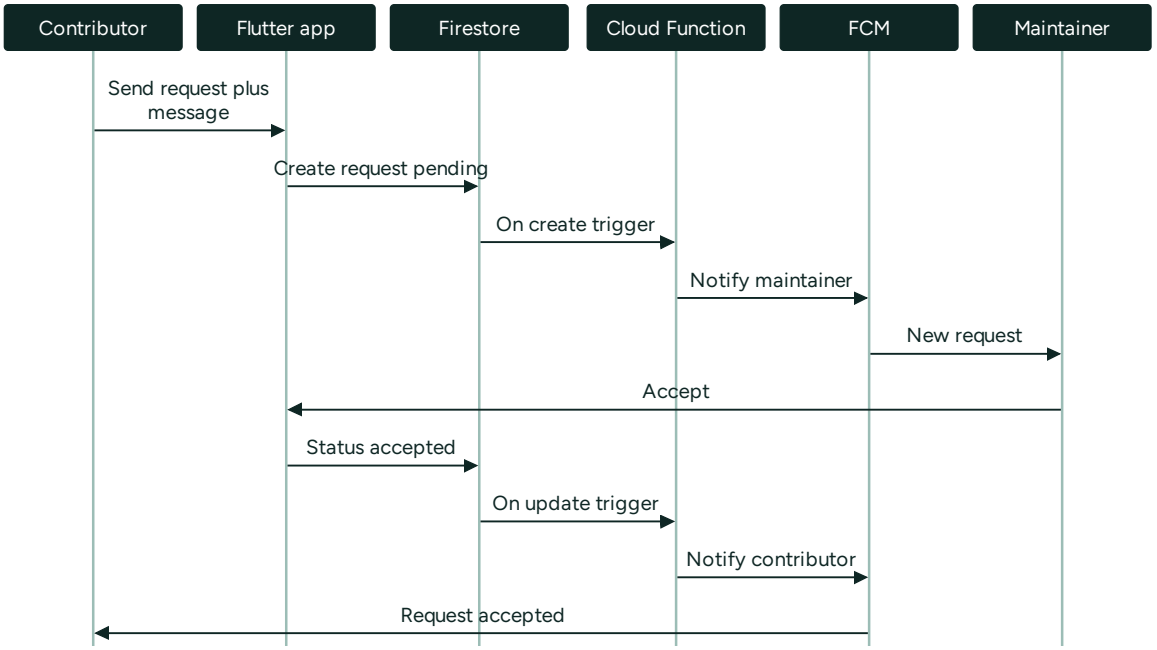


Figure 9. Request and FCM sequence. Planned.

3.6 Activity — core user journey

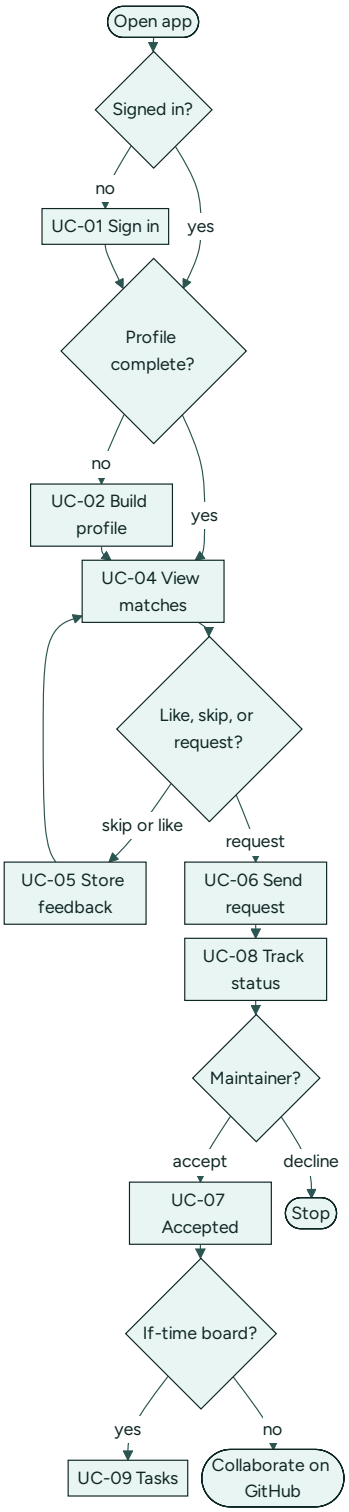


Figure 10. Activity diagram for the core journey.

3.7 State diagrams

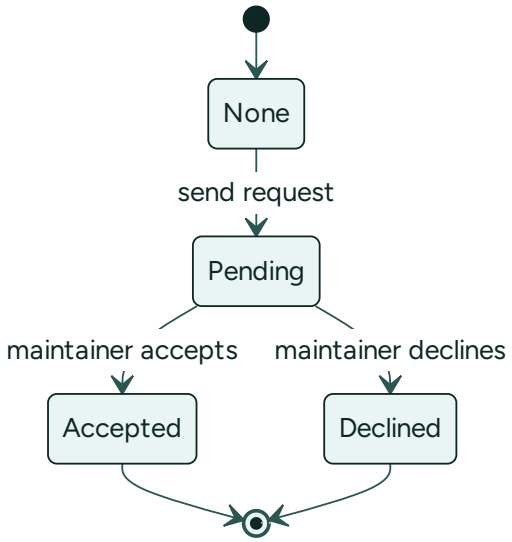


Figure 11. Collaboration request states. One request per user–project.

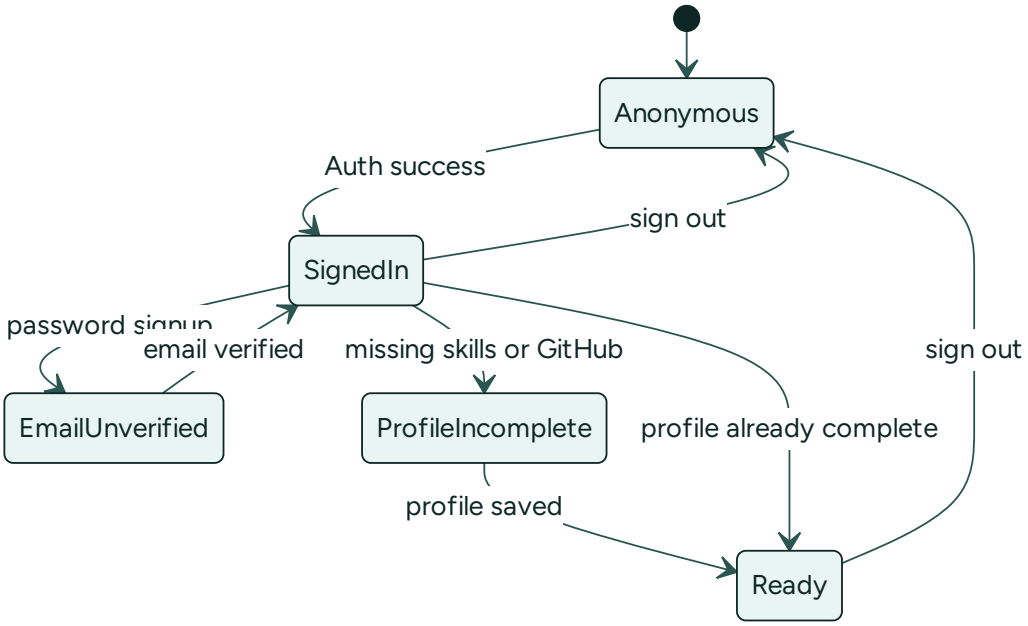


Figure 12. Auth and profile readiness. Email verification is **Implemented** for password signup.

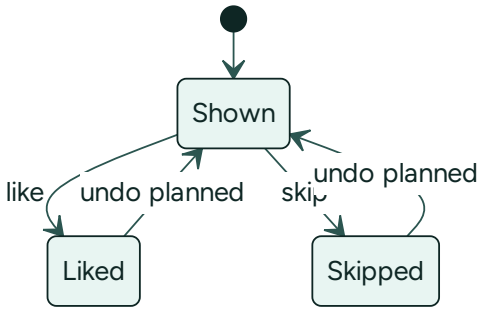


Figure 13. Match-card feedback states.

3.8 Class diagram

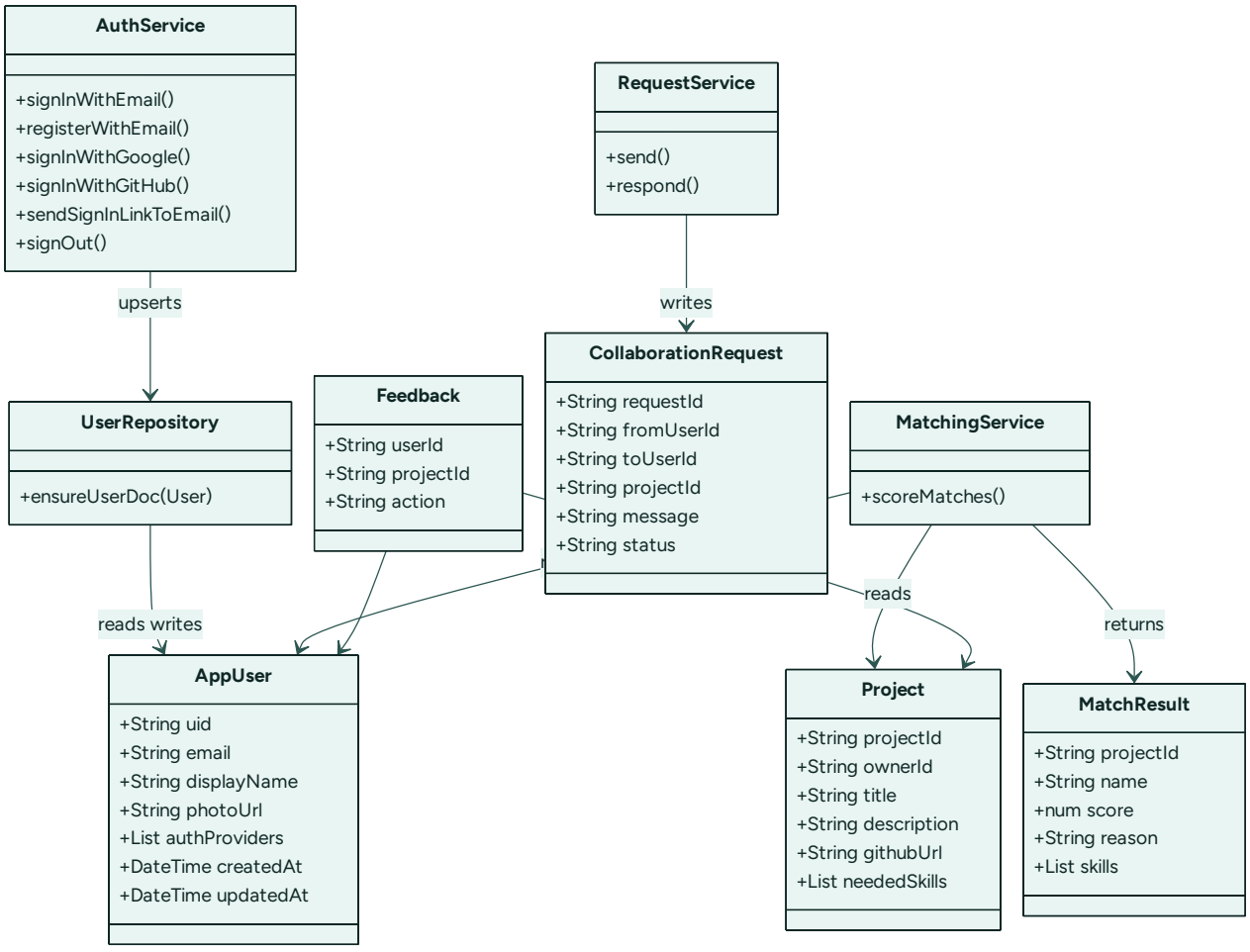


Figure 14. Class view. `AppUser`, `AuthService`, and `UserRepository` are **Implemented**. `Project`, `MatchResult`, `Feedback`, `CollaborationRequest`, `MatchingService`, and `RequestService` are **Planned**. `MatchResult` fields are the frozen feed contract.

PART 4

UI/UX design and prototyping

4.1 Wireframes and mockups

Screens are not redrawn in this report. The design references are:

- Interactive prototype: `docs/index.html` (Team, Flow, and Prototype tabs)
- Brand tokens and logo use: `docs/DevSync Brand Guide.html`
- Flutter design-system gallery route: `/design-system`

Representative frames in the prototype: Welcome, Login, Match feed, Project detail, Requests, Profile.

4.2 Screen inventory

Bottom navigation (planned product): Matches · Discover · Projects · Requests · Profile.

Screen	Owner layer	Status
Welcome, Login, Register, email link, verify email	Donia · Auth	Implemented
Forgot password, OTP verify	Donia · Auth	UI present; **demo only**
Home shell (Feed / Projects / Requests / Profile)	Yousef · Shell	Implemented as stubs
Match feed, fallback feed	Emad · ranking; Yousef · like/skip	Planned
Discover, project list, edit, detail	Bishay · Projects	Planned
Request sheet, inbox, detail, alerts	Yousef · Requests	Planned
Profile, edit profile, GitHub connect	Saga · Profile	Planned
Settings	Donia · account/security; Saga · GitHub/About via PR	Planned
Delete account	Donia	Planned
Task board	Donia	If-time
Prompt dashboard	Emad	If-time

Prototype-only screens (hub chat, community, AI Studio, insights) may appear in `index.html`. They are **not** committed FRs and are not required.

4.3 UI/UX guidelines

Token	Value
Teal	`#2EE0C8`
Forest / on-teal	`#0E2624`
Mint	`#E8F5F2`
Dark surface	`#153532`
Headings	Bricolage Grotesque
Body	Figtree
Primary button height	56 dp

Token	Value
Minimum touch target	48 dp (NFR-13)
Contrast	4.5:1 WCAG AA (NFR-13)
Icons	Lucide via `AppIcons` / `DsIcon`
Logo	Two paths merging into one node

Flavors (planned): Free = dark card, teal icon. Pro = teal card, dark icon. Not a paid product for this course.

Motion: Bishay applies animation only after the screen owner merges to `develop`. Motion PRs change animation only — no Firestore, no business logic, no copy. Screen owner reviews within 24 hours.

4.4 Shared-screen handoffs

Ownership on shared screens:

Screen	Split
Match feed	Emad owns score, reason, list, and card. Yousef owns like/skip buttons and feedback documents.
Settings	Donia is screen owner (session, sign-out, permissions). Saga PRs only GitHub + About / build.
Animations	Bishay; owner-reviewed motion PRs.

PART 5

System deployment and integration

5.1 Technology stack

Layer	Technology	Status
Client	Flutter, Dart `^3.12.2`	**Implemented**
UI	`google_fonts`, `lucide_icons_flutter`, `Ds*` widgets	**Implemented**
Auth	`firebase_core`, `firebase_auth`, `google_sign_in`	**Implemented**
Database	`cloud_firestore`	**Implemented** for `users` only
Deep links	`app_links` (email-link continue URL)	**Implemented**
Local prefs	`shared_preferences`	**Implemented**
Functions	`cloud_functions`	**Planned** — not in pubspec
Push	`firebase_messaging`	**Planned** — not in pubspec
HTTP	GitHub REST client	**Planned** — no `http` / `dio` yet
LLM	Gemini Flash-Lite via Function secret	**Planned**
Process	GitHub PRs, Jira Scrum	In use
Distribution	GitHub Releases or Firebase App Distribution	**Planned**

Android `applicationId` today: `com.example.devsync_open_source_collaboration_app`. No `productFlavors`, no minify, debug signing on release — all **Planned** to change.

5.2 Deployment diagram

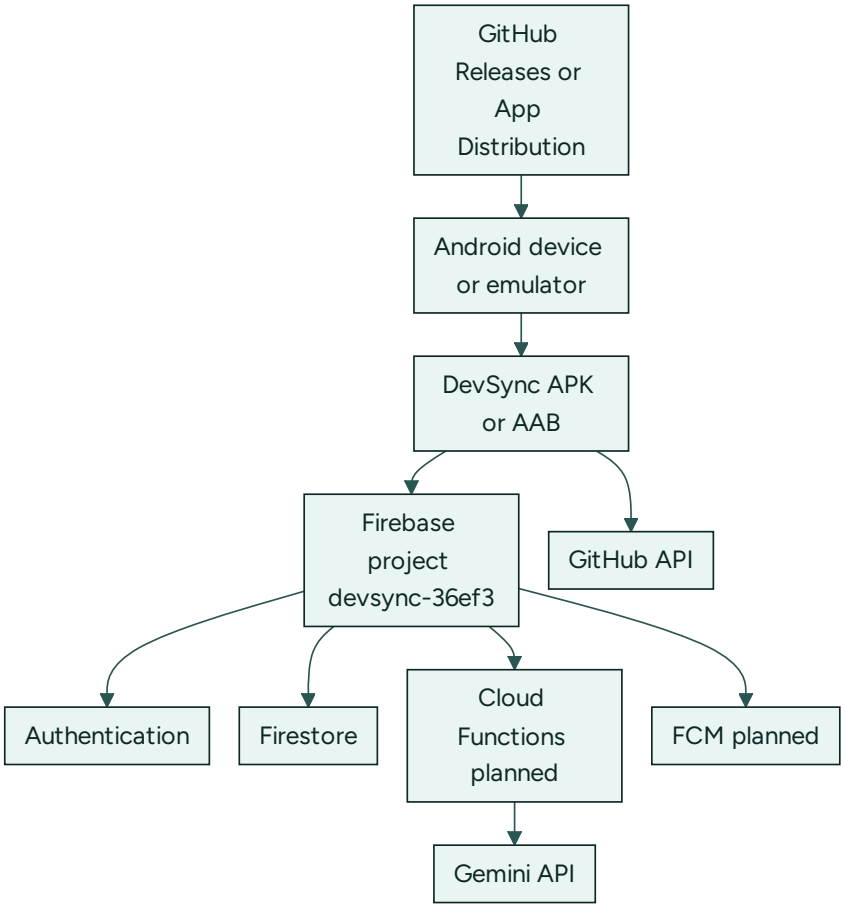


Figure 15. Deployment. No Play Store. Course delivery is a signed, obfuscated install link.

5.3 Component diagram

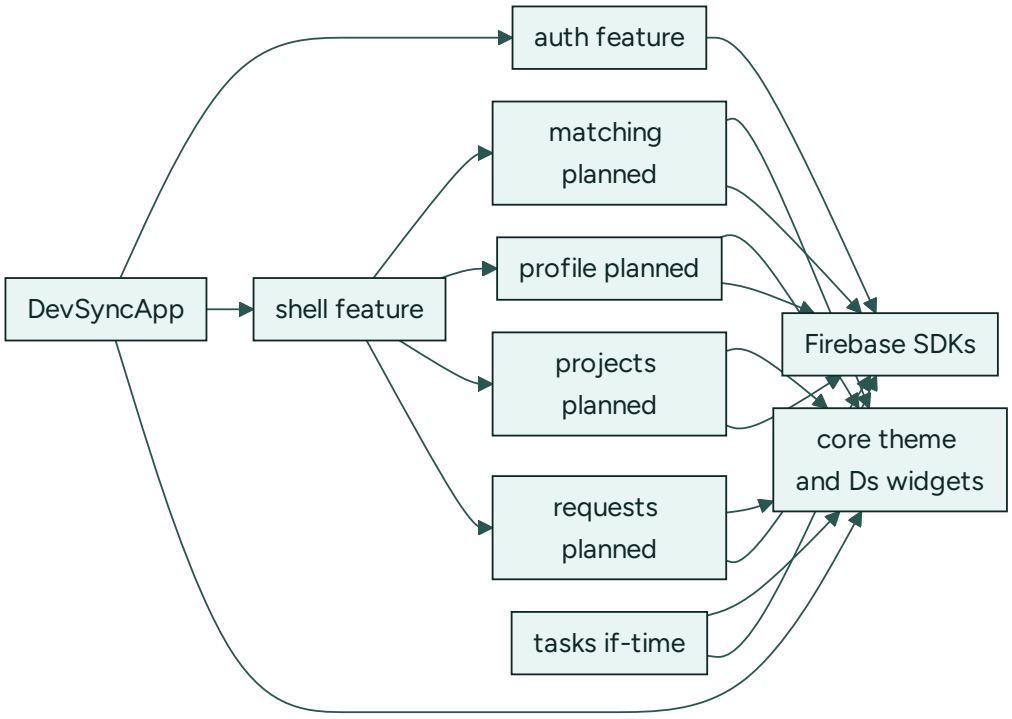


Figure 16. Components. `matching`, `profile`, `projects`, `requests` are empty folders. `tasks` is if-time.

PART 6

Additional deliverables

6.1 API documentation

There is **no OpenAPI spec** in the repo. The interfaces are described below.

Firebase Auth and Firestore (client SDKs, implemented)

- Auth methods in `AuthService`: email/password, Google, GitHub OAuth (`read:user`, `user:email`), email link to `https://devsync-36ef3.firebaseapp.com/finishSignIn`.
- Firestore: `users/{uid}` upsert via `UserRepository.ensureUserDoc`.

Planned callable `scoreMatches`

Transport	Firebase callable or HTTPS Cloud Function
Auth	Required Firebase ID token
Input	Current user id; server loads profile and candidate projects (the client never handles the LLM key)
Output	Array of frozen feed items: `projectId`, `name`, `score`, `reason`, `skills[]`
Errors	Timeout or model failure → skills fallback; client shows a banner
Secret	Gemini API key stays a Function secret

Planned FCM triggers

- On `requests` create: notify maintainer.
- On `requests` status change: notify contributor.
- If-time: task assign and status.

Planned GitHub public REST

- Client-side fetch of public profile data only (NFR-12).
- Cache on the user document; batch refresh (FR-07).
- No private repos. Handle rate limits.

Gemini

- Called **only** from Cloud Functions.
- Primary model: Gemini Flash-Lite; name in config so it can be swapped.
- Eval: one hosted open-source model on the evaluation set. Do not self-host.

6.2 Testing and validation

Layer	What	Status
Widget	Welcome carousel and Login navigation	Implemented in `test/widget_test.dart`
Unit	Matching logic, data models, Firestore rules	Planned
Integration	Sign-in, feed, requests	Planned
Prompt eval	Labelled set in repo; compare prompt versions	Planned
FCM	Delivery script with timestamps (NFR-03)	Planned
Devices	Two real Android phones and one emulator	Planned
Process	Author is not the tester before merge; 70% matching-logic coverage	Planned

Saga owns the test and security report and maps each FR to a test case. CI (analyze + tests on every PR) is Planned; no `.github/workflows` yet.

User acceptance (pilot): 20 or more users; 60% complete a profile; 10 or more requests; 50% of top-5 matches rated relevant; relevance up 15% from first to final prompt.

6.3 Deployment strategy

Item	Plan
Hosting	Firebase project `devsync-36ef3`; confirm Blaze before W3 for Functions
Environments	Dev and prod Firebase projects; budget alerts
Pipeline	Planned GitHub Actions: analyze + tests on PRs; no direct pushes to `main`
Branches	`main`, `develop`, `feature/`, `fix/`, `release/`; Jira key in commit subjects
First release build	Obfuscated build by **12 Nov 2026**
Final delivery	Free and Pro signed builds, `.aab`, install link, 5-line README guide by **30 Nov 2026**
Store	Not in scope
Scale	Pilot 20+ users; cache matches; limit LLM calls per user

APPENDIX A

Team and backup

Member	Role	Core pts	Backup
Donia Alhosin	Project manager; foundation, auth, security rules	12 + PM	Yousef
Bishay Talaat	Design and brand; projects, animations, presentation	7	Emad
Yousef Mohy	App architecture; shell, CI, requests, FCM, like/skip	10	Saga
Emad Hany	AI and prompts; matching, feed, evaluation	8	Donia
Saga Maher	Quality and release; profile, builds, reports	10	Bishay

Backup chain: Donia → Yousef → Saga → Bishay → Emad → Donia.

Appendix B Flutter routes today

From `lib/app/app_routes.dart`: `/`, `/welcome`, `/login`, `/register`, `/forgot-password`, `/code-sign-in`, `/email-link-sent`, `/verify-code`, `/verify-email`, `/design-system`, `/home`, `/feed`, `/projects`, `/projects/detail`, `/profile`, `/requests`. Feature routes behind `/home` are stubs.